Distributed Systems
COMP 212

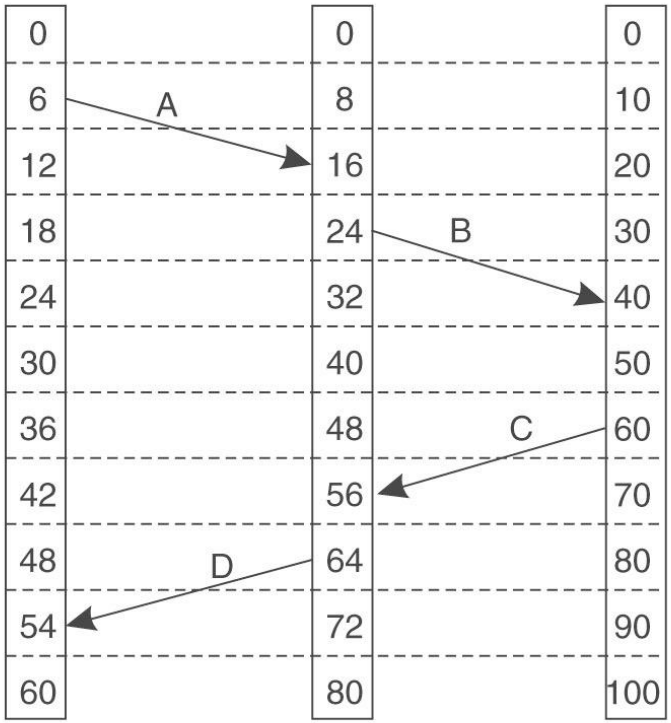
Revision 2

Othon Michail

UNIVERSITY OF
LIVERPOOL

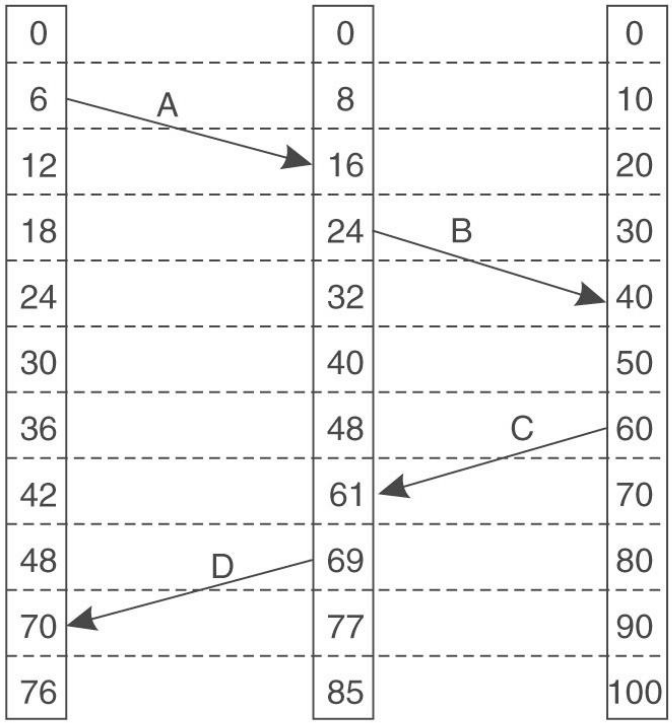
Synchronisation

- How would Lamport's algorithm synchronise the clocks in the following scenario?



(a)

- How would Lamport's algorithm synchronise the clocks in the following scenario?



(b)

- Imagine that each machine in a Distributed System has its own internal extremely accurate clock and all clocks are identical. In order to achieve clock synchronisation, we synchronise all clocks initially, once and for all. Is this a sufficient solution and why?

- Imagine that each machine in a Distributed System has its own internal extremely accurate clock and all clocks are identical. In order to achieve clock synchronisation, we synchronise all clocks initially, once and for all. Is this a sufficient solution and why?

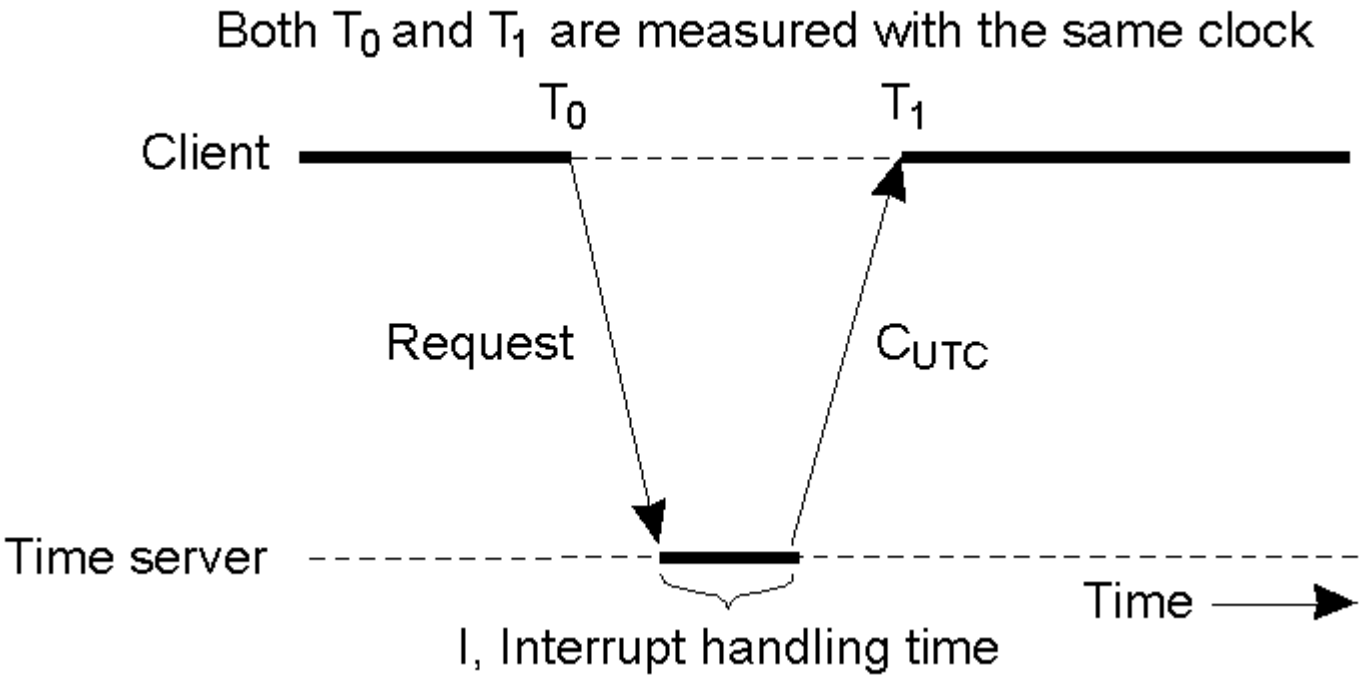
- No
- Even if clocks on all computers in a DS are set to the same time, due to clock skew, their clocks will eventually vary quite significantly unless corrections are applied, and this holds for all types of clocks

- Imagine that we are using Cristian's algorithm to synchronise clocks in a Distributed System.
  1. Describe Cristian's algorithm.
  2. If the time-server B responds to a client A with a time T_B less than the current time on A's clock, is it ok for A to set its clock immediately to T_B?

- Imagine that we are using Cristian's algorithm to synchronise clocks in a Distributed System.
  1. Describe Cristian's algorithm.
  2. If the time-server B responds to a client A with a time T_B less than the current time on A's clock, is it ok for A to set its clock immediately to T_B and why?

  1. Next slide
  2. No:
     - Time should never go backwards as this could lead to serious local inconsistencies (e.g. file system; new versions of files having smaller timestamps than old versions)
     - Instead, the change should be implemented gradually by delaying the local clock until B's clock catches it up

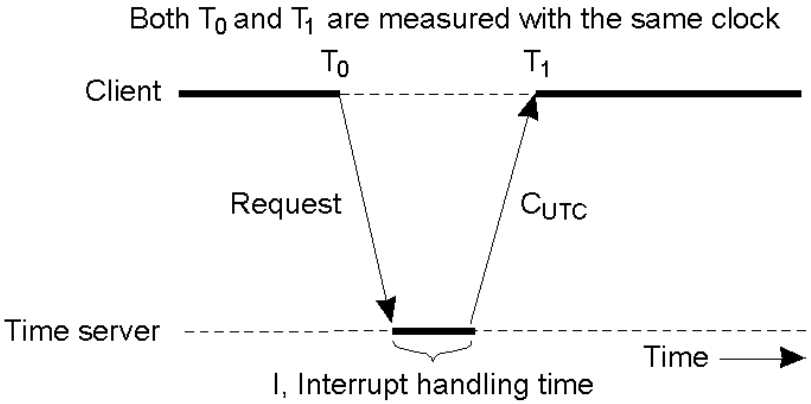
Clock Sync. Algorithm: Cristian's



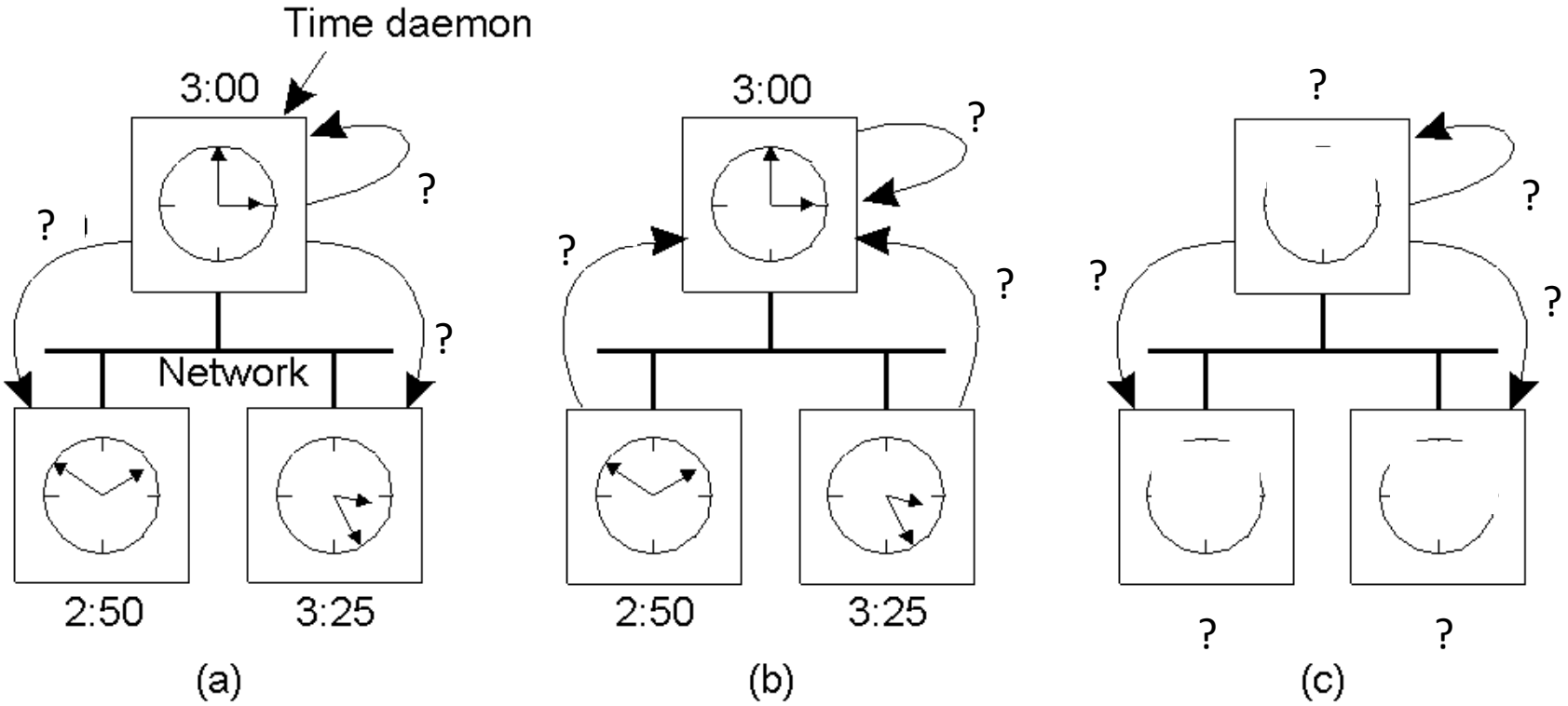
1. Every computer periodically asks the "time server" for the current time
2. The server responds ASAP with the current time C_{UTC}
3. The client sets its clock to C_{UTC}

Problems

- Major problem: if time from time server is less than the client – resulting in time running backwards on the client! (Which cannot happen – time does not go backwards).



- Introduce changes gradually
- Minor problem: results from the delay introduced by the network request/response: latency
- *Best estimate* $(T_1-T_0)/2$
- If the interrupt handling time, I, is known, $(T_1-T_0-I)/2$
- **Use series of measurements**

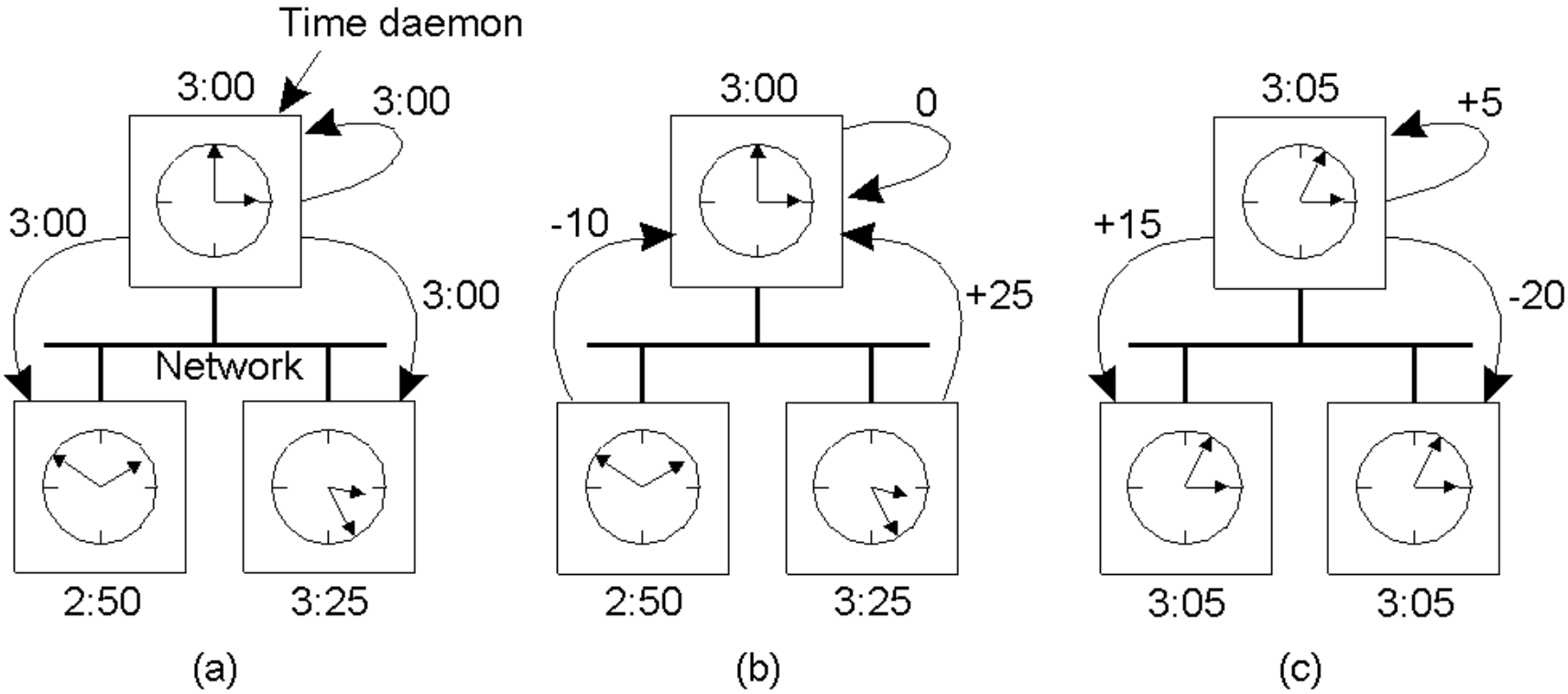


- Fill in all the missing messages transmitted by the Berkeley clock synchronisation algorithm in this setting and the new values of the 3 clocks after synchronisation

Berkeley Algorithm

- An algorithm for internal synchronisation of a group of computers
- A master polls to collect clock values from the others (slaves)
- The master uses round trip times to estimate the slaves' clock values
- It takes an average
- It sends the required adjustment to the slaves (better than sending the time which depends on the round trip time)
- If master fails, can elect a new master to take over

The Berkeley Clock Sync. Algorithm



- Clocks that are running fast, are slowed down
- Clocks running slow, jump forward

Transactions

- What are the 2 main functionalities that transactions offer?

Transactions

1. Protect a shared resource against simultaneous access by concurrent processes
   - This can be also achieved by mutual exclusion algorithms
2. Allow a process to access and modify multiple data in a single atomic operation
   - Benefit: when half-success is not acceptable, everything can be restored as it never occurred

- Explain the ACID (standing for Atomic, Consistent, Isolated, and Durable) characteristics that must be satisfied by a transaction

ACID

- The four key transaction characteristics

Transactions are:

- **Atomic**: The transaction is considered to be one thing, even though it may be made of up many different parts
- **Consistent**: "Invariants" that held before the transaction must also hold after its successful execution
- **Isolated**: If multiple transactions run at the same time, they must not interfere with each other. To the system, it should look like the two (or more) transactions are executed sequentially (i.e., that they are serializable).
- **Durable**: Once a transaction commits, any changes are permanent

- Explain what we mean when we say that a transaction is nested. Mention a possible disadvantage of this type of transaction.

- Explain what we mean when we say that a transaction is nested. Mention a possible disadvantage of this type of transaction.

- Nested Transactions: a main, parent transaction spawns child sub-transactions to do the real work
- Disadvantage: problems result when a sub-transaction commits and then the parent aborts the main transaction. Things get messy but still manageable. Which characteristic of transactions is violated in this case?

- Explain what a private workspace and a writeahead log are and why they are useful for transactions.

- Explain what a private workspace and a writeahead log are and why they are useful for transactions.

- Private Workspace: Until the transaction either commits or aborts, all of the reads and writes go to the private workspace. The original data are available to other processes during the transaction.
- Writeahead log: Files are modified in place, but a record is written to a log prior to that. Only changes the file, after the log has been written successfully
  - If the transaction aborts, the log can be used to rollback to the original state
- Both are useful techniques for undoing changes in case of an abort

Mutual Exclusion

- Using an example, demonstrate how a deadlock can arise in transaction processing

- Using an example, demonstrate how a deadlock can arise in transaction processing

- A transaction T1 acquires a lock on an object X, whereas a different transaction T2 acquires a lock on a different object Y. However, T1 is waiting T2 to release the lock on Y, whereas T2 is waiting T1 to release the lock on X. This results in a deadlock.

- Explain what is the difference between centralised and distributed mutual exclusion
- Give an example execution of the centralised mutual exclusion algorithm

DS Mutual Exclusion: Techniques

Two major approaches:

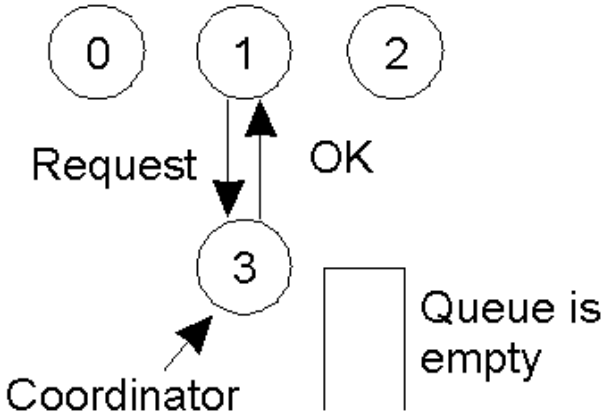
- **Centralised**: a single coordinator controls whether a process can enter a critical region
- **Distributed**: the group confers to determine whether or not it is safe for a process to enter a critical region

Centralised Algorithm

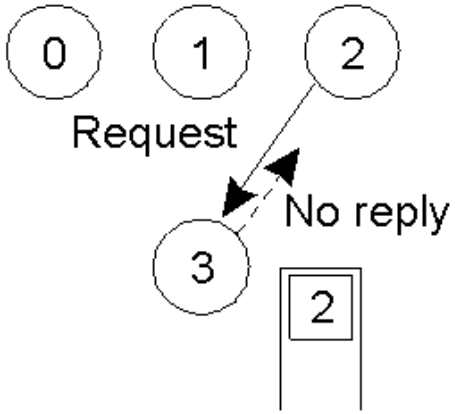
a) Process 1 asks the coordinator for permission to enter a critical region. Permission is granted.

b) Process 2 asks for permission to enter the same region. No reply.

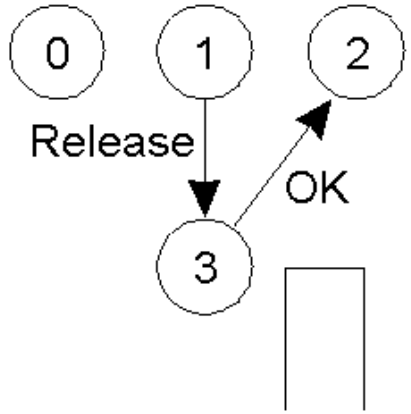
c) When Process 1 quits the critical region, it tells the coordinator, which then replies to Process 2



(a)

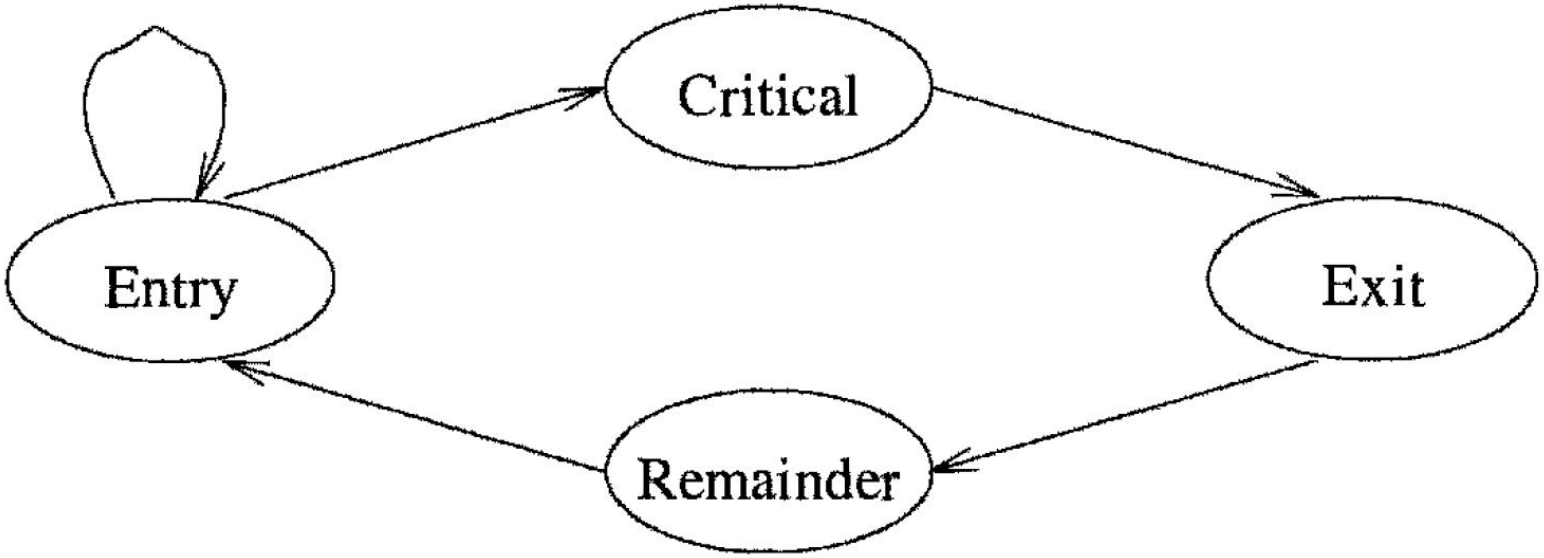


(b)



(c)

- Explain all the terms that appear in the following figure. Which of these sections are handled by a mutual exclusion algorithm?



General Structure of Solutions

Programs are partitioned into the following sections:

- Entry (trying): the code executed in preparation for entering the critical section
- Critical: the code to be protected from concurrent execution
- Exit: the code executed on leaving the critical section
- Remainder: the rest of the code

- A mutual exclusion algorithm consists of code for the entry and exit sections
  - Should work no matter what the other two sections implement

Replication

- Why it is important to replicate data in a Distributed System?

Why Replicate Data?

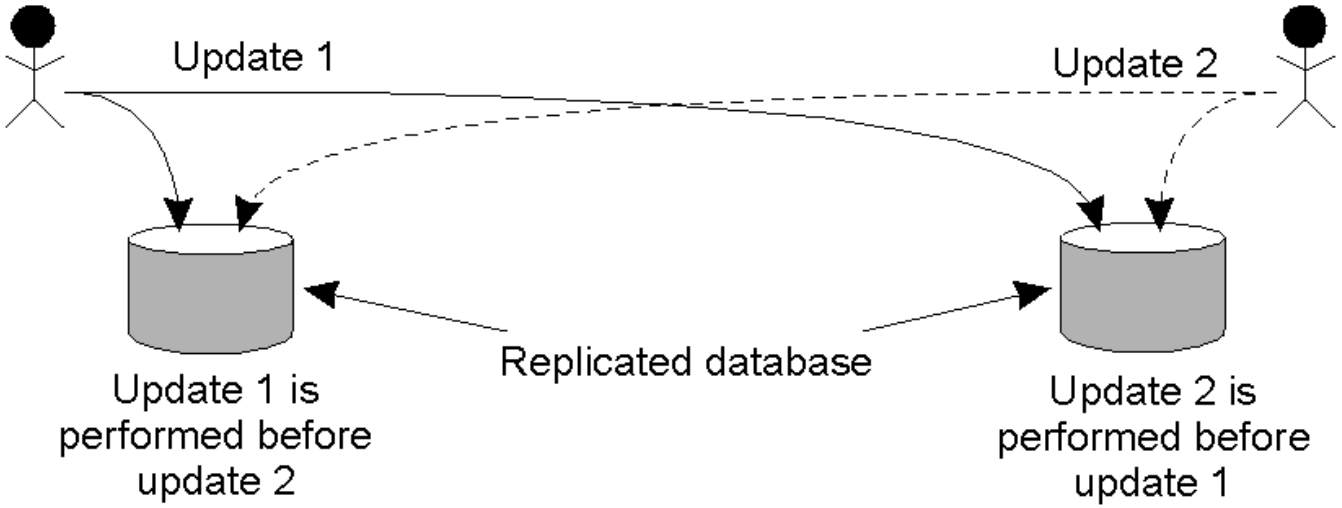
- Enhance reliability
  - While at least one server has not crashed, the service can be supplied
  - Protection against corrupted data (the majority of the copies is expected to be correct)
- Improve performance
  - Increasing the #clients would overload a single server
    - e.g., several web servers can have the same DNS name and the servers are selected in turn to share the load
  - Placing copies of data in the proximity of processes using them

More on Replication

- Replicas allow remote sites to continue working in the event of local failures
- Possible to protect against data corruption
- Replicas allow data to reside close to where it is used
- This directly supports the distributed systems' goal of enhanced scalability
- Even a large number of replicated "local" systems can improve performance
  - think of clusters

- Give a example of inconsistency of replicated data that can be severe

What Can Go Wrong

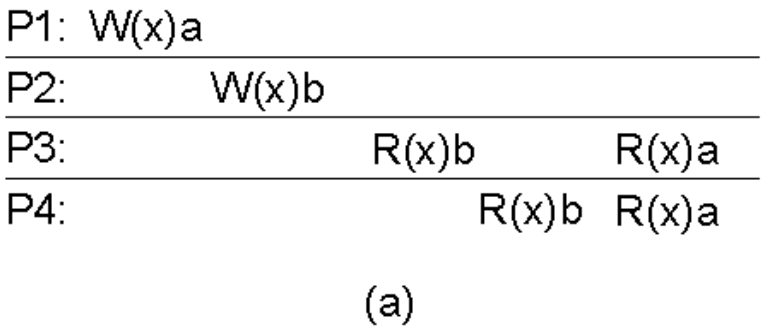


- Updating a replicated database: Update 1 adds £ 100 to an account, Update 2 calculates and adds 1% interest to the same account
- Due to network delays, the updates may come in different order!
- Inconsistent state: The same account has two different balances!

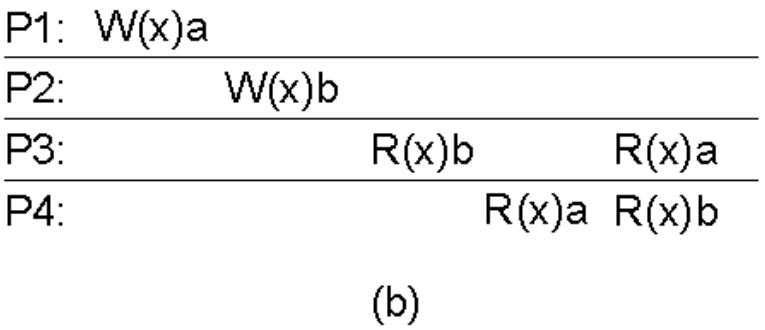
- Explain what we mean by sequential consistency

Example: Sequential Consistency

- *All* processes see the same interleaving set of operations, regardless of what that interleaving is



(a)



(b)

a) A sequentially consistent data-store
   - the "first" write occurred *after* the "second" on all replicas

b) A data-store that is not sequentially consistent
   - it appears the writes have occurred in a non-sequential order, and this is NOT allowed

- Describe the push and pull based approaches of update propagation in distributed replicas and mention an example of a hybrid approach

Push vs. Pull Protocols

1. ***Push-based/Server-based Approach***: sent "automatically" by server, the client does not request the update
   - Useful when a high degree of consistency is needed
   - Often used between permanent and server-initiated replicas

2. ***Pull-based/Client-based Approach***: used by client caches (e.g., browsers), updates are requested by the client from the server
   - No request, no update!

- A hybrid approach: leases

Fault Tolerance

- Name three different types of faults (in terms of a fault's frequency) and for each one of them mention at least one practical example

Main Types of Faults

- **Transient** fault: occurs once and then disappears
  - A bird flying through a beam of a microwave transmitter
  - Some bits might get lost but a retransmission will probably work
- **Intermittent** fault: may reappear again and again
  - A loose contact on a connector
- **Permanent** fault: continues to exist until the faulty component is replaced
  - burn-out chips, software bugs, disk head crashes

- What is a crash and what a Byzantine failure? Which one of the two is considered harder to deal with?

- What is a crash and what a Byzantine failure? Which one of the two is considered harder to deal with?

- Crash failure: A server halts, but is working correctly until it halts
- Byzantine failure: A server may produce arbitrary responses at arbitrary times (even malicious)
- Byzantine is in general worse due to its unpredictable behaviour

- Give the three main types of redundancy and explain each one of them

Failure Masking by Redundancy

- Strategy: if we cannot avoid failures then better hide them from other processes and/or users
  - using redundancy

Three main types:

1. Information Redundancy
   - Add extra bits to allow for error detection/recovery
   - e.g., parity bits, Hamming codes

2. Time Redundancy
   - Perform operation and, if required, perform it again.
   - Think about how transactions work (BEGIN/END/COMMIT/ABORT)
   - Well suited for transient and intermittent faults

3. Physical Redundancy
   - Add extra (duplicate) hardware and/or software components to the system
   - Think of replication

- Explain the difference between the forward and backward recovery strategies from failures and mention some of their disadvantages

- Explain the difference between the forward and backward recovery strategies from failures and mention some of their disadvantages

1. Backward Recovery: return the system to some previous correct state (using *checkpoints*), then continue executing
   - Checkpointing (can be very expensive, especially when errors are very rare)
   - No guarantee that we won't meet the same error again
   - Some operations cannot be rolled back

2. Forward Recovery: bring the system into a correct state, from which it can then continue to execute
   - all potential errors need to be accounted for up-front so that the system knows how to "fix" them

Security

- What is the main difference between symmetric and asymmetric cryptosystems? Which one of the two is also called public-key and why?

- What is the main difference between symmetric and asymmetric cryptosystems? Which one of the two is also called public-key and why?

- In symmetric, both the sender and the receiver use the same key for encryption/decryption while in asymmetric they use different keys
- Asymmetric, because one of the two keys can be made public

- Assume that a polynomial-time (i.e., efficient) algorithm was found, for computing the prime factors of integers. Which encryption algorithm would no longer be safe to use in this case?

- Assume that a polynomial-time (i.e., efficient) algorithm was found, for computing the prime factors of integers. Which encryption algorithm would no longer be safe to use in this case?

- The RSA algorithm because it constructs the keys based on large prime numbers, relying on the fact that no efficient method is known to find the prime factors of large numbers